Draw the Line: An SVM Showdown

Liran Ma, Zhipeng Cai

1 SVM

Key Idea: Map original data into a higher-dimensional space to make it separable.

1.1 Motivating Example

Imagine you are at a football game, with fans from two teams sitting in a stadium. Each group is mixed together. You want to draw a line (hyperplane) to separate fans from the two teams, such that the **closest fans from each team are as far from the line as possible** (max margin).

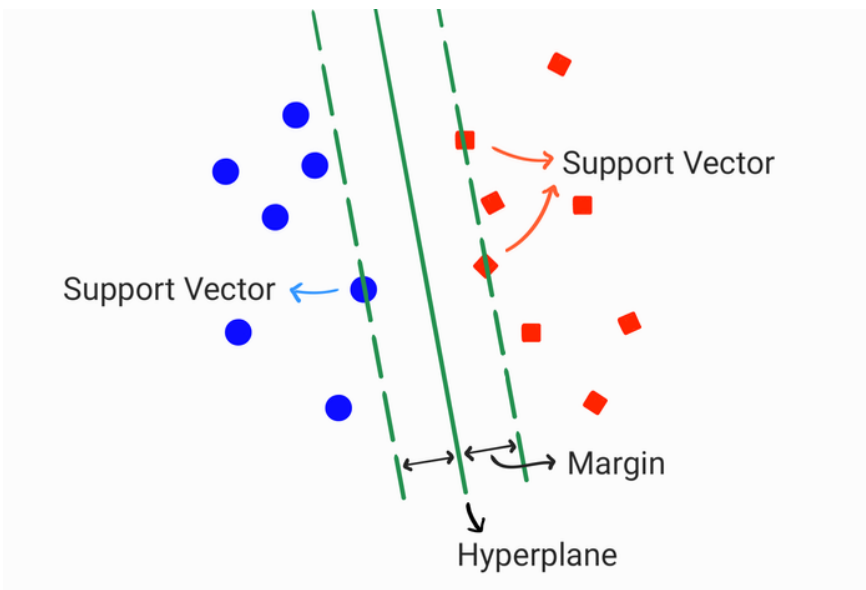


Figure 1: SVM illustration: Red Team Fans vs Blue Team Fans

1.2 What is SVM?

- SVM is a supervised learning model for classification and regression (mainly used for classification).

Supervised vs Unsupervised Learning

Supervised learning: The model is trained on labeled data (each example has a correct answer). Typical tasks: classification, regression.
Example: Email spam detection, where emails are labeled as "spam" or "not spam".

Unsupervised learning: The model is trained on unlabeled data and tries to find patterns or structure by itself. Typical tasks: clustering, dimensionality reduction.
Example: Grouping customers by purchasing behavior without knowing their categories.

- The goal is to find a hyperplane that separates data points from different classes, **maximizing the margin** between them.
- **Support vectors**: Data points closest to the hyperplane, which are most critical for defining the margin.

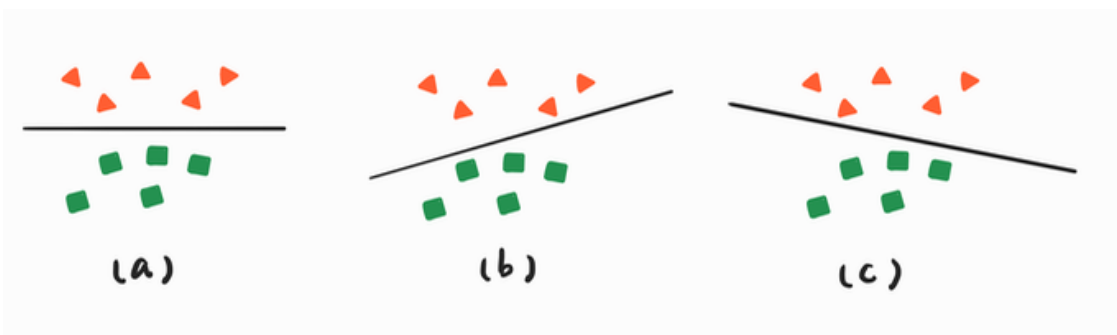


Figure 2: Three possible separating lines (a, b, c) for the two classes.

Question:
Which line (a, b, or c) do you think is the best separator between the two classes? Why?

1.3 How does SVM work?

Now, let's see how **SVM** works: it automatically finds the line (hyperplane) that **maximizes the margin** between the two classes!

- In 2D, the hyperplane is a line; in 3D, it's a plane; in higher dimensions, it's called a hyperplane.
- Mathematically: For each data point (x_i, y_i), $y_i \in \{-1, +1\}$, require $y_i(w \cdot x_i + b) \geq 1$. Here, b is the *intercept* (or bias) of the hyperplane, not an error term. It shifts the hyperplane up or down in space.
- The objective is to maximize the margin, which is equivalent to minimizing $\|w\|^2$. ($\|w\|$ is the Euclidean norm (length) of the weight vector w. Minimizing $\|w\|^2$ is equivalent to maximizing the margin (distance between the hyperplane and the closest points)).

Questions:
Why y_i is -1 and $+1$? Why $y_i(w \cdot x_i + b) \geq 1$?

1.4 Limitation: Linearly Separable Case

While SVM can perfectly separate classes when the data are linearly separable, **not all datasets can be separated by a straight line (or hyperplane)** in the original feature space. This is a major limitation of the basic linear SVM.

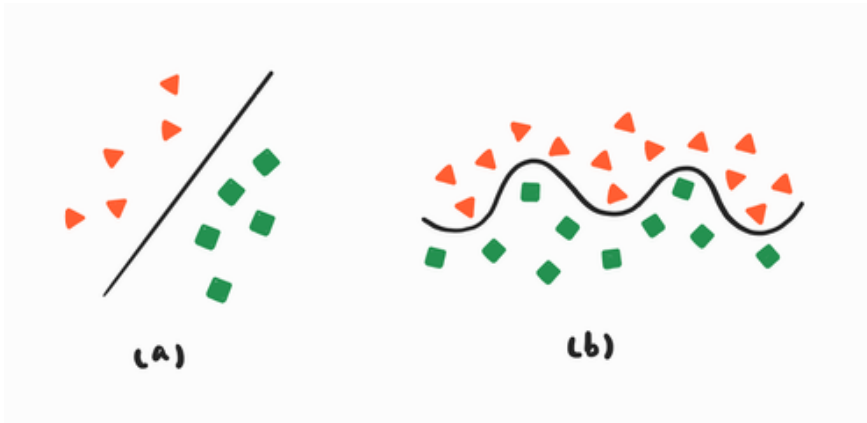


Figure 3: (a) Data that can be separated by a straight line (**linearly separable**).
(b) Data that cannot be separated by any straight line (**not linearly separable**); a nonlinear boundary is needed.

To address this, SVM introduces the **kernel trick**, which allows the algorithm to find a separating hyperplane in a higher-dimensional space, even if the data are not linearly separable in the original space.

1.5 Kernel

Kernel trick: Use a mapping function ϕ to project data into a higher-dimensional feature space, where the data becomes linearly separable. The kernel function computes the inner product in this higher-dimensional space without explicitly calculating the mapping.

Mapping function (Kernel function): The choice of kernel determines the shape of the decision boundary. Commonly used kernels include:

1.5.1 Linear kernel

$K(x, x') = x \cdot x'$

Suitable for linearly separable data.

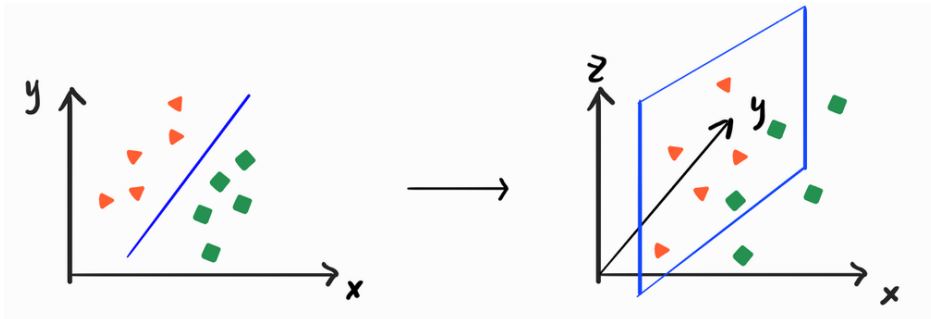


Figure 4: Example: Using AI to classify patients with (orange) and without (green) heart disease using a linear kernel. In the left 2D plot, the blue line is the decision boundary. In the right 3D plot, this decision boundary becomes a plane after mapping the data into a higher-dimensional space.

1.5.2 Polynomial kernel

$$K(x, x') = (x \cdot x' + c)^d$$

Suitable for data with polynomial decision boundaries.

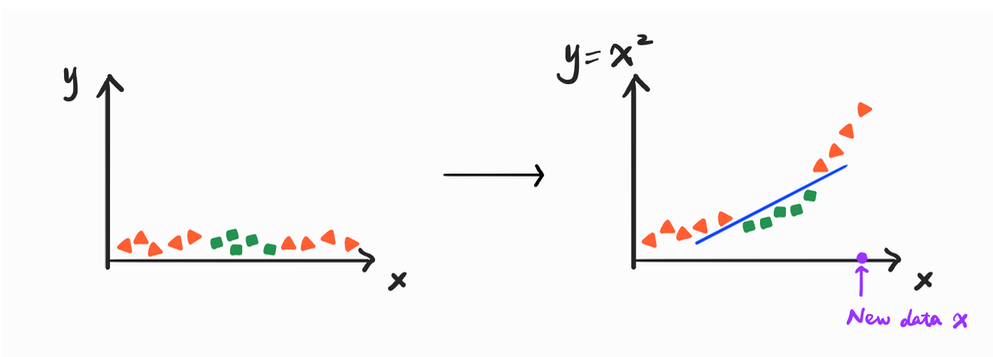


Figure 5: Example: Using AI to classify patients with (orange) and without (green) heart disease using a polynomial kernel

Here, the kernel function is $y = x^2$. This transformation maps the original features into a higher-dimensional space where the two classes become linearly separable.

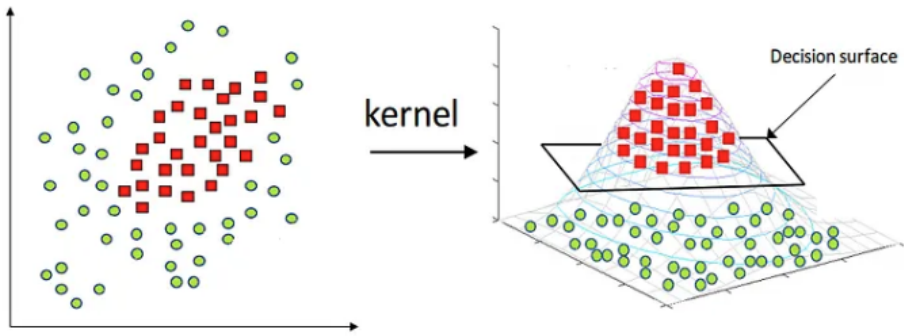
After training the model, we can classify new data points. The purple point in the right figure represents a new patient.

Question: Based on the trained decision boundary in the transformed space, would you classify this new patient as having heart disease or healthy? Give your reasoning.

1.5.3 Radial Basis Function (RBF) kernel

$$K(x, x') = \exp\left(-\gamma \|x - x'\|^2\right)$$

Suitable for data with complex, nonlinear decision boundaries.



Source: Support Vector Machine (S.V.M.) – Classifiers and Kernels by Apurv Jain (Medium).

Figure 6: Example: Using AI to classify two classes with a radial basis function (RBF) kernel. In the left 2D plot, the red squares and green circles represent two classes that are not linearly separable. The RBF kernel maps the data into a higher-dimensional space (right plot), where the classes can be separated by a curved *decision surface* (shown in blue). The central peak represents regions with a high decision function value for the red class, allowing the SVM to handle complex, nonlinear decision boundaries.

1.6 Lab: SVM on Stroke Risk Prediction Dataset

As a hands-on example, we will use the **Stroke Risk Prediction Dataset (Kaggle, Mahatir Ahmed Tusher)** to build a binary classifier (stroke vs. no stroke). We will split the data, standardize features, train an RBF-kernel SVM, and evaluate performance.

1.6.1 Data Cleaning and Preparation

This section details the initial steps taken to load, clean, and prepare the raw stroke risk dataset for machine learning.

- **Load the Data:** The process begins by loading the dataset from a CSV file into a pandas DataFrame.

```
# 1) Load CSV ----------------------------------------------------------------
CSV_PATH = Path("/Users/.../stroke_risk_dataset.csv") #change the path
df = pd.read_csv(CSV_PATH)
```

Listing 1: Loading the CSV file.

- **Clean Column Names** Column names are standardized to remove spaces and special characters, facilitating easier access.

```
# 2) Clean column names
df.columns = [re.sub(r"[^0-9A-Za-z]+", "_", c).strip("_") for c in df.columns]
```

Listing 2: Standardizing all column names.

- **Define Target and Remove Leaky Data** The target variable is defined, and any columns that could cause data leakage are removed.

```python
# 3) Define target and drop leakage columns ---------------------------------
TARGET = "At_Risk_Binary"
LEAK_COLS = ["Stroke_Risk", "Stroke_Risk_", "Stroke_Risk__"]
leaks_present = [c for c in LEAK_COLS if c in df.columns]
X = df.drop(columns=[c for c in leaks_present + [TARGET] if c in df.columns]).
    copy()
y = df[TARGET].astype(int)
```

Listing 3: Separating features and target; removing data leaks.

1.6.2 Model Training and Evaluation

This section covers the process of splitting the data, building the model pipeline, tuning hyperparameters, and evaluating the final model.

- **Train / Test Split** The preprocessed data is split into training and testing sets. ‘stratify=y‘ ensures both sets have the same proportion of target classes.

```python
# 5) Train / test split ----------------------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

Listing 4: Splitting data into training and testing sets.

- **Model Pipeline Setup** A ‘Pipeline‘ chains the data scaling and the SVM model together. This is crucial for preventing data leakage during cross-validation.

```python
# 6) Pipeline: Standardize + SVM (RBF kernel) ------------------------------
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC(kernel="rbf", class_weight="balanced"))
])
```

Listing 5: Defining the SVM pipeline with a scaler and RBF kernel.

- **Hyperparameter Tuning with GridSearchCV** ‘GridSearchCV‘ systematically searches for the best combination of SVM hyperparameters to maximize the F1-score.

```python
# 7) Hyperparameter search -------------------------------------------------
param_grid = {
    "svm__C": [0.5, 1, 5, 10],
    "svm__gamma": ["scale", 0.1, 0.01, 0.001]
}
grid = GridSearchCV(pipe, param_grid, cv=5, n_jobs=-1, scoring="f1")
grid.fit(X_train, y_train)
best = grid.best_estimator_
```

Listing 6: Searching for optimal hyperparameters.

- **Model Evaluation** The best model is evaluated on the unseen test set using accuracy, a confusion matrix, and a classification report.

```python
# 8) Evaluation ------------------------------------------------------------
y_pred = best.predict(X_test)
print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("Confusion matrix:\n", confusion_matrix(y_test, y_pred))
```

```python
print("Classification report:\n", classification_report(y_test, y_pred, digits=3))
```

Listing 7: Evaluating the best model on the test set.

```
Accuracy: 0.9942857142857143
Confusion matrix:
 [[4909    2]
 [  78 9011]]
Classification report:
               precision    recall  f1-score   support

           0      0.984     1.000     0.992      4911
           1      1.000     0.991     0.996      9089

    accuracy                          0.994     14000
   macro avg      0.992     0.996     0.994     14000
weighted avg      0.994     0.994     0.994     14000
```

Figure 7: Evaluation metrics of the best SVM model on the test set. The accuracy is **99.43%**, with the confusion matrix showing only **80 misclassifications** out of 14,000 samples. The classification report indicates high precision, recall, and F1-score for both classes, with slightly higher recall for class 0 and perfect precision for class 1.

1.6.3 Visualizing Performance with ROC Curve

The ROC curve is plotted to visualize the model's performance across different classification thresholds.

```python
# 9) ROC curve ---------------------------------------------------------------
RocCurveDisplay.from_estimator(best, X_test, y_test)
plt.title("SVM (RBF) ROC - Stroke Risk (Binary)")
plt.show()
```

Listing 8: Plotting the ROC curve for the final model.

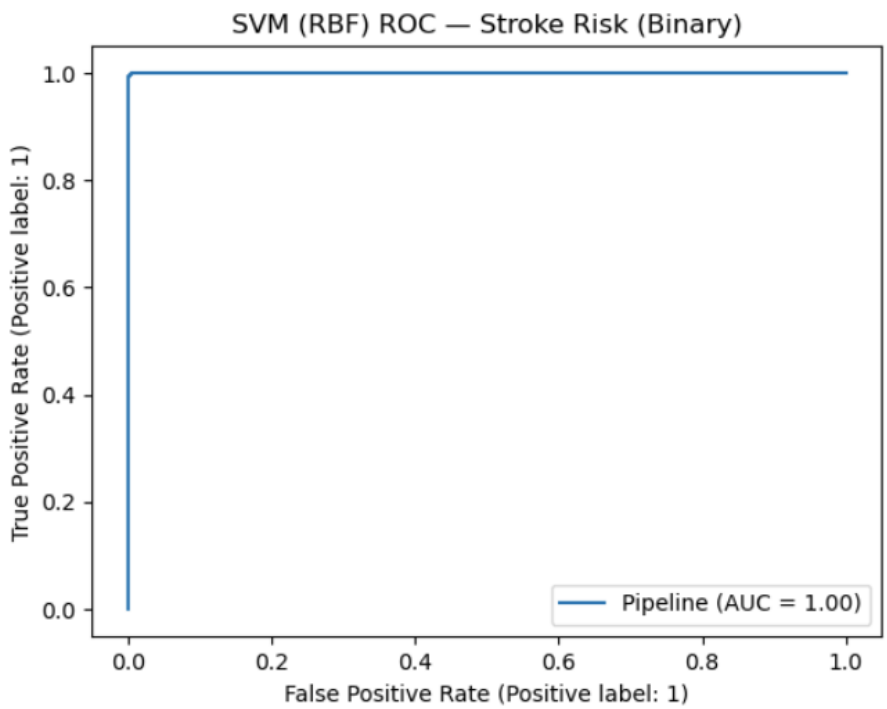


Figure 8: ROC curve of the best SVM (RBF) model on the test set. The curve is almost a perfect square, and the area under the curve (AUC) is **1.00**, indicating near-perfect discrimination between positive and negative classes. This means the model can almost always correctly distinguish between patients at risk and not at risk of stroke.

1.6.4 Feature Importance with a Linear SVM

A separate Linear SVM is trained to estimate which features have the most impact on the prediction, as the RBF kernel lacks direct interpretability.

```
# 10) Linear SVM for feature importance -------------------------------------
lin_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC(kernel="linear", class_weight="balanced"))
])
lin_clf.fit(X_train, y_train)
coefs = lin_clf.named_steps["svm"].coef_.ravel()
feat_importance = pd.Series(coefs, index=X.columns).sort_values(key=np.abs,
    ascending=False)
print("\nTop features by absolute linear weight:\n", feat_importance.head(10))
```

Listing 9: Using a Linear SVM to estimate feature importance.

```
Top features by absolute linear weight:
 Age                          21.051679
Fatigue_Weakness              4.999028
Persistent_Cough              4.999023
Snoring_Sleep_Apnea           4.999021
Chest_Discomfort_Activity     4.999013
Irregular_Heartbeat           4.999007
Anxiety_Feeling_of_Doom       4.999003
Swelling_Edema                4.998995
Cold_Hands_Feet               4.998987
Chest_Pain                    4.998983
dtype: float64
```

Figure 9: Top features ranked by the absolute value of their linear SVM coefficients. The feature **Age** has the highest absolute weight (21.05), indicating it has the strongest influence on the decision boundary. Other features such as *Fatigue_Weakness*, *Persistent_Cough*, and *Snoring_Sleep_Apnea* also contribute notably. Positive coefficients push the classification toward the positive (at-risk) class, while negative coefficients push toward the negative (not-at-risk) class.

For a linear SVM, the decision boundary is defined by the hyperplane:

$$\mathbf{w}^\top \mathbf{x} + b = 0$$

where $\mathbf{w}$ is the vector of learned feature weights and b is the intercept term. For this model, using the top 10 features, the hyperplane equation can be expressed as:

$$\begin{aligned}
&21.0517\, x_{\text{Age}} + 4.9990\, x_{\text{Fatigue_Weakness}} + 4.9990\, x_{\text{Persistent_Cough}} \\
&+ 4.9990\, x_{\text{Snoring_Sleep_Apnea}} + 4.9990\, x_{\text{Chest_Discomfort_Activity}} + 4.9990\, x_{\text{Irregular_Heartbeat}} \\
&+ 4.9990\, x_{\text{Anxiety_Feeling_of_Doom}} + 4.9990\, x_{\text{Swelling_Edema}} + 4.9989\, x_{\text{Cold_Hands_Feet}} \\
&+ 4.9989\, x_{\text{Chest_Pain}} + b = 0
\end{aligned}$$

Find b: $b =$ _____

Tip

b (the intercept term) can be obtained in Python using:

```python
b = float(lin_clf.named_steps["svm"].intercept_)
```

Listing 10: Extracting the intercept term from the linear SVM.

1.7 Advantages of SVM

- Works well with high-dimensional data.
- Effective when the number of dimensions > number of samples.
- Flexible kernel choices allow handling various data structures.
- Robust to overfitting if margin is large.

1.8 Disadvantages of SVM

- Training time can be long with large data sets.
- Sensitive to the choice of kernel and hyperparameters.
- Not suitable for data sets with significant class overlap.
- Does not directly provide probability estimates.

1.9 Summary

- SVM finds the hyperplane with the largest margin to separate classes.
- Kernel trick enables SVM to handle nonlinearly separable data.
- Common kernels: linear, polynomial, RBF.
- Suitable for high-dimensional, small-to-moderate-sized data sets.